

Bitwise-Reproducible Reduction Across Cluster Reshape

A Compiler/Fabric ABI

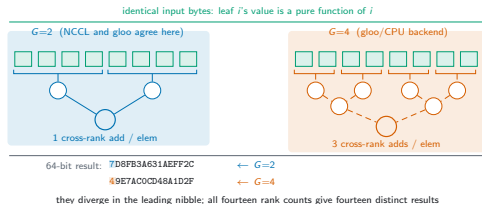
The same sum, twice, different answers

Reduce the *identical* bytes on a cluster.

Reshape it. Reduce again.

- Floating-point addition is not associative.
- Changing the worker count changes *which operands pair*.
- So the bit pattern changes.

We measured it rather than asserting it: across fourteen world sizes, fourteen distinct roots on identical input.



Why this matters

- Debugging: a run that cannot be reproduced cannot be bisected.
- Regulation and audit: “same input, same output” is often a requirement.
- Elasticity: cloud clusters reshape *by design*.

The usual answers

Fix the worker count (gives up elasticity), or sort/compensate at every step (costly, and still order-dependent under reshape).

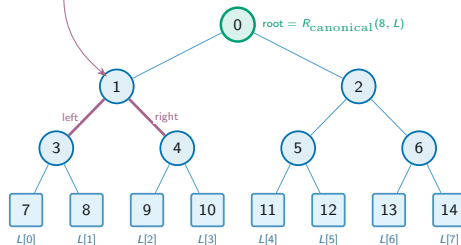
The idea: name one order, and make it a contract

Fix a *canonical* reduction tree $\text{FBT}(N)$ — heap-indexed, leaves in ascending index order.

The result is defined by the *tree*, not by the machine that walks it.

The compiler must prove it emits that tree; the fabric must refuse to execute anything else.

operand binding at $v=1$ (Def. 3): exactly one `COMPUTE`, with $\text{addr.a}=\text{dmem}(3)$ and $\text{addr.b}=\text{dmem}(4)$ — a pure function of the node v . Neither G , nor topology, mapping, refinement or schedule appears in it.



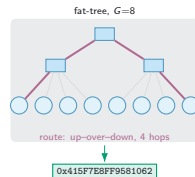
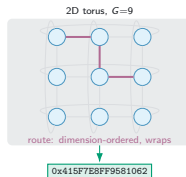
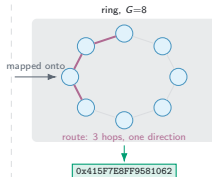
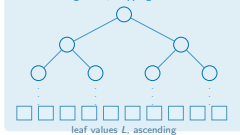
$\text{FBT}(N)$ has $2N-1$ nodes; children of n at $2n+1$ and $2n+2$; internal $\{0, \dots, N-2\}$ are circles and leaves $\{N-1, \dots, 2N-2\}$ squares, so every internal node has exactly two children.

Leaf enumeration $\text{idx}(n) = n - (N-1)$: the k -th smallest *heap* index takes $L[k]$. That is left-to-right *visual* order only when N is a power of two.

Internal post-order $\pi_g = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$; campaign root at $N=8$ is $0x40E1948E978D4FDF$ (Table I).

Claim 1 — the root does not depend on the topology

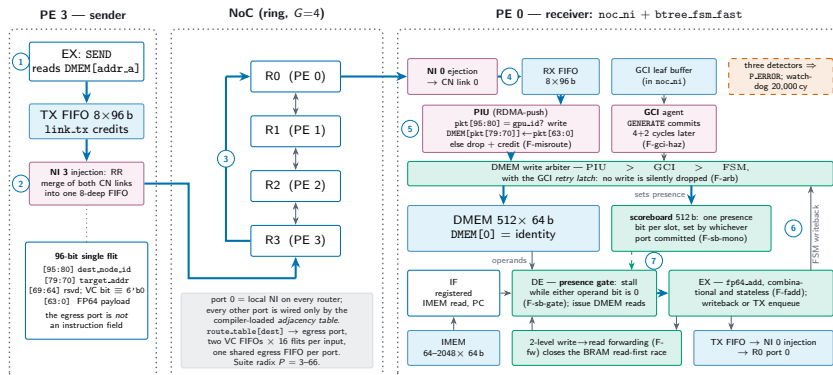
FBT(N) at $N=128$: operand binding is a function of tree-node identity alone — it names no G , no topology, no assignment, mapping or schedule



The routes differ; the root does not. Over the whole campaign (2,822 settled simulations, 22 non-settling) no configuration produced a finite root differing from $R_{\text{canonical}}(N, L)$. At $N=128$ alone the same root survives six (G , topology) points, $G=4$ on a linear chain to $G=128$ on a torus, over 371 passing hardware configurations with 0 divergent.

Routes differ. The reduction DAG does not. The 64-bit root does not.

The fabric enforces it below the software

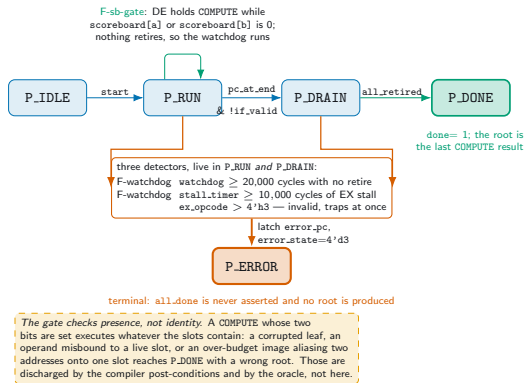


- 1 SEND forms the flit 2 the NI merges the two CN links into one injection FIFO 3 one hop: route-table lookup, RR arbitration, crossbar
4 ejection to CN link 0 5 the PIU checks dest_gpu_id and writes DMEM itself (no RECV) 6 presence bit set 7 the stalled COMPUTE leaves DE

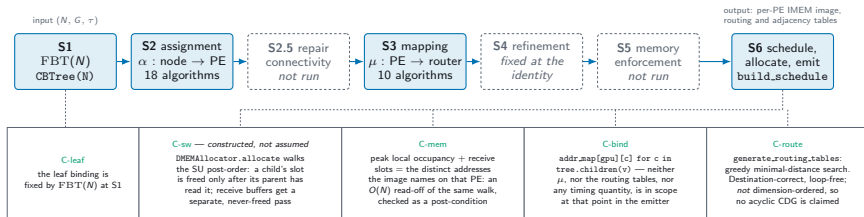
Property 2 — fail-stop, not wrong-answer

A missing or misrouted operand must *halt* the run.

It must never silently produce a different root — a wrong answer that looks like a right one is the failure mode that matters.

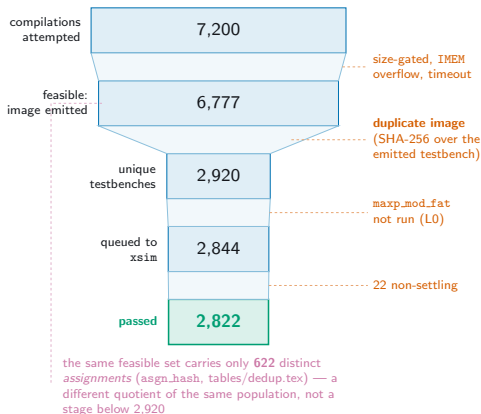


The compiler discharges the obligations



Why two of the skipped passes stay skipped: running `repair_connectivity` moves a median 50.8% of all tree nodes and inflates peak per-PE load by a median factor of 1.86, so we report the unrepaired pipeline and check the bound directly; and μ reaches the emitted image only through the NOP padding the cost model places — recompiling five representative points under all ten mappings yields one-three distinct images.

The campaign

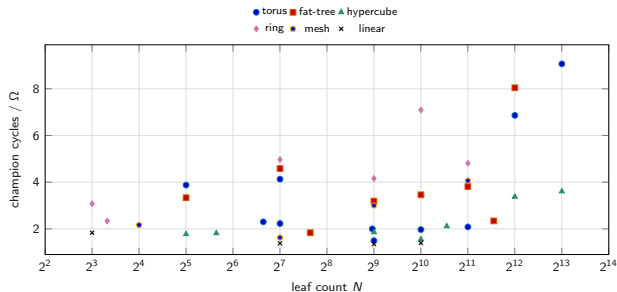


- 2,844 compilations attempted
- 2,822 passing hardware runs
- 39 of 40 instances hardware-validated
- 6 topologies, N from 8 to 8192

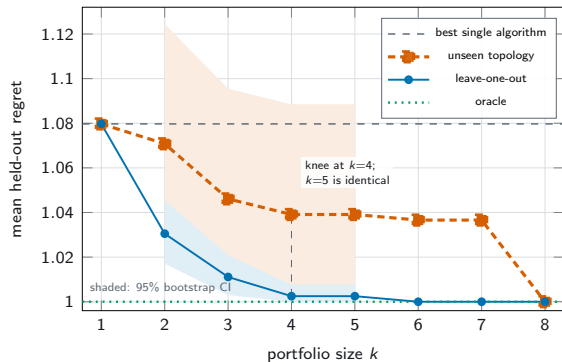
The 40th instance elaborates completely; the simulator emits no kernel for it. We report that, and assert no cause.

Cross-topology invariance holds

- Every passing configuration yields a single root per instance.
- Mutation testing: the oracle is not vacuous — reordering *does* change the root, at twelve of the sixteen leaf counts.



Which algorithm should you actually run?

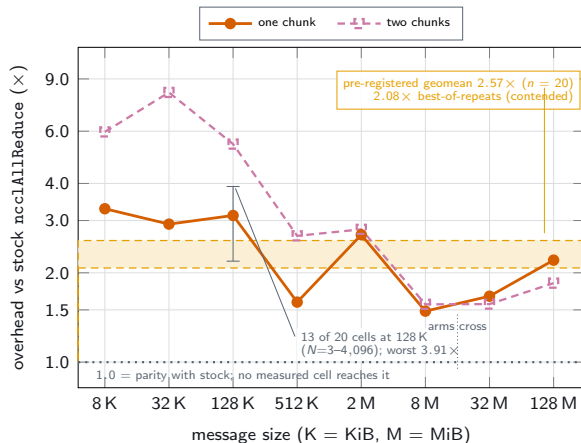


A portfolio of **four** configurations, run and minimised over:

- 1.0391 mean regret on a topology it *never saw* (95% CI [1.0025, 1.0885])
- vs 1.0488 for a rule that *conditions* on topology

Ignoring topology beats conditioning on it.

What it costs, in software



Pre-registered, and unfavourable:

- $2.57\times$ stock `ncclAllReduce` at $G=2$
- provisional — measured under GPU contention
- about half of it is algorithmic at $G=2$, not an implementation defect

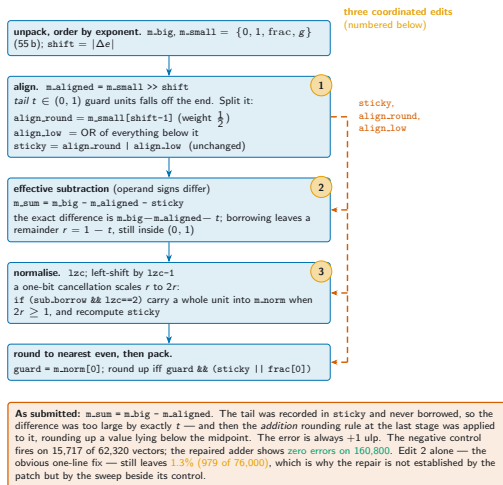
We pre-registered this comparison before running it, and report the branch we hit.

What we got wrong

Our own FP64 adder had a defect on effective subtraction — and our Python oracle was a transcription of that RTL, so it agreed.

Found by scoring against references *external* to the design: exact rationals and the host FPU.

zero errors on 160,800 vectors after repair, beside a control that fires on 15,717 of 62,320.



Limitations we are not hiding

- No silicon: RTL simulation of a single-chip fabric.
- No measured same-fabric η at non-power-of-two scale. At power-of-two, $\eta = 1$ is a *theorem*, not a measurement.
- The suite confounds N and G ; no cell-level recommendation is derivable.
- The software export is slower than stock NCCL, and we say by how much.

Takeaway

Reproducibility across reshape is achievable as an *interface contract*: the compiler proves the order, the fabric refuses anything else, and the result stops depending on the shape of the machine.

- Bitwise-identical roots across six topologies.
- Fail-stop rather than silently wrong.
- A four-configuration portfolio that transfers to unseen topologies.